

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

The rapid advancement in wireless communication and embedded systems technology has revolutionized the concept of home automation. Smart Home Automation is an innovative approach that integrates technology with household appliances to provide enhanced control, comfort, and security.

This project utilizes the HC-05 Bluetooth module paired with the STM32 microcontroller to create a cost-effective, efficient, and user-friendly system for managing home devices remotely. Bluetooth technology is widely adopted due to its low power consumption, short-range communication capability, and ease of interfacing with smartphones, making it an ideal choice for smart home applications.

This system allows users to remotely switch appliances on or off via a Bluetooth-enabled mobile device, providing convenience and improved energy management. Smart home systems are increasingly significant in modern residential settings as they contribute to energy savings, better security, and improved quality of life.

Smart home systems are becoming increasingly important in modern residential settings as they contribute to energy savings, better security, and improved quality of life. This project's primary goal is to develop a working prototype that showcases the capability of embedded systems to communicate wirelessly and control real-world devices effectively. It offers insights into integrating hardware and software to facilitate intuitive and reliable home automation, reflecting current trends in IoT and wireless control technologies.

It offers insights into integrating hardware and software to facilitate intuitive and reliable home automation, reflecting current trends in IoT and wireless control technologies. Overall, this project not only highlights the technical feasibility but also serves as a stepping stone towards more complex and scalable smart home solutions.

1.2 Problem Statement

“Many homes still rely on manual operation of electrical appliances, which can be inconvenient and inefficient”.

- In many traditional households, the control and management of electrical appliances are performed manually, which often leads to multiple inefficiencies. For instance, residents may forget to turn off lights, fans, or other devices when they leave a room or the house, resulting in unnecessary energy consumption and increased electricity bills.
- This manual approach also lacks convenience, especially for elderly individuals or those with mobility limitations, as physically reaching switches can be difficult.
- Furthermore, the absence of centralized or remote control systems limits the flexibility and responsiveness needed in modern homes. Existing commercial smart home automation solutions often come with high costs, complex installation procedures, and require robust internet infrastructure, which may not be available or affordable for every household.
- Many systems relying on Wi-Fi or cloud-based platforms may also pose security and privacy concerns due to data transmission over the internet.
- In contrast, Bluetooth technology offers a more localized and secure communication channel but has been underutilized in this domain. This project aims to solve these issues by developing a low-cost, simple-to-use, and efficient Smart Home Automation system leveraging the HC-05 Bluetooth module and STM32 microcontroller.
- This project aims to address the lack of a simple, cost-effective solution for remote home appliance control using Bluetooth technology and an embedded microcontroller and solves the problem of inconvenient manual control of home appliances by enabling users to wirelessly operate devices using a mobile application.
- The system enables real-time remote control of home appliances through a smartphone without dependence on internet connectivity, thereby improving energy efficiency, convenience, and usability for everyday users.

1.3 Objectives

1. To design and implement a wireless system for controlling home appliances such as fans and LED lights using Bluetooth technology and a mobile application.
2. To enhance user convenience and energy efficiency by allowing remote activation and deactivation of devices through the STM32 microcontroller and relay interface.
3. To create a reliable and user-friendly home automation prototype that demonstrates the integration of wireless communication, embedded control, and practical power management using readily available electronic components.

CHAPTER 2

METHODOLOGY

2.1 METHODOLOGY

The methodology chapter details the step-by-step process followed for designing, developing, and implementing the Smart Home Automation system using the HC-05 Bluetooth module and STM32 microcontroller. The approach combines hardware integration, embedded software development, and user interface design to achieve seamless wireless control over home appliances.

2.1.1 System Design:

The system architecture consists of three main components: the smartphone application, the Bluetooth communication module (HC-05), and the microcontroller unit (STM32). The smartphone serves as the user interface, sending control commands via Bluetooth to the HC-05 module, which acts as a serial transmitter and receiver. The STM32 microcontroller receives these commands and actuates the respective home appliances through relay modules.

2.1.2 Hardware Implementation:

The hardware setup includes connecting the HC-05 Bluetooth module to the STM32 microcontroller using UART communication pins. The STM32 is interfaced with relay driver circuits to control high-power household devices safely. Power supply units were configured to provide stable voltage levels to the microcontroller and connected peripherals. A breadboard prototype was initially used for circuit validation, followed by a custom PCB design to enhance reliability and compactness.

2.1.3 Testing and Validation:

Extensive testing was conducted to verify the system's functionality, responsiveness, and reliability. Various appliances were connected and controlled remotely through the smartphone under different scenarios to ensure command accuracy and minimal latency. The system's robustness was assessed under continuous operation for extended periods.

2.1.4 System Enhancement and Scalability:

Though initially designed for a limited number of appliances, the system architecture allows easy scalability by adding more relay channels and expanding the command set in the firmware. Future enhancements could include integrating sensors for automation based on environmental conditions and adding security features for data protection.

2.2 Block Diagram

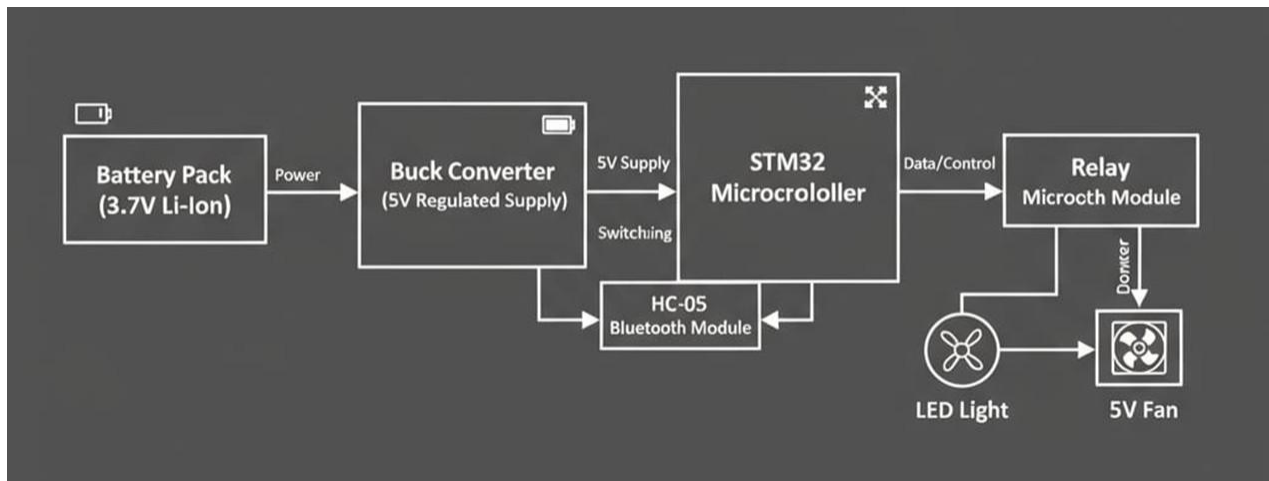


Figure 2.2[a]: Block Diagram

- **Battery Pack (3.7V Li-Ion):** This is the primary power source for the system, providing the initial energy required.
- **Buck Converter (5V Regulated Supply):** The battery's 3.7V supply is stepped up and regulated to a stable 5V supply using a buck converter. This ensures that components like the microcontroller and Bluetooth module receive a consistent voltage for proper operation.
- **STM32 Microcontroller:** This is the central control unit of the system. It receives switching commands from the Bluetooth module and processes them to control connected appliances via the relay module. It manages the data flow and overall coordination of the system.
- **HC-05 Bluetooth Module:** Connected to the microcontroller via UART or similar communication protocol, this module handles wireless communication with the user's smartphone. It receives command inputs remotely, which are then relayed to the microcontroller.

- Relay Microtech Module: Controlled by the STM32 microcontroller through data/control signals, the relays act as electronic switches to turn on or off the connected appliances like the LED light and 5V fan.
- LED Light and 5V Fan: These represent the load or appliances controlled in your smart home setup, which can be remotely operated via the system.

In summary, the battery pack powers the entire system through the buck converter. The microcontroller acts on Bluetooth commands received via the HC-05 module to switch the relays, which in turn control the appliances. This setup enables remote and wireless automation of home devices efficiently.

2.3 Working of the Circuit

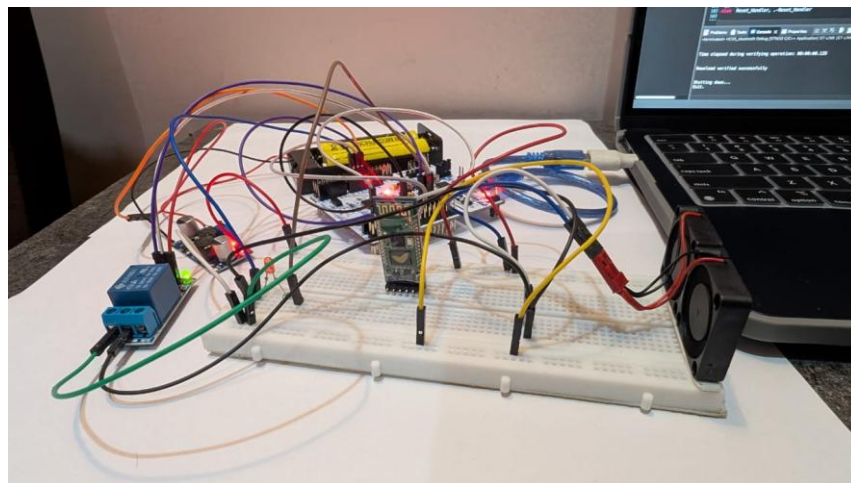


Figure 2.3[a]: circuit connection

2.3.1 Power Supply Initialization

- The system is powered by a 3.7V Li-Ion battery pack.
- The battery voltage is stepped up and regulated to a stable 5V supply using the buck converter.
- This 5V regulated output powers the STM32 microcontroller, HC-05 Bluetooth module, and relay module circuits, ensuring all components operate reliably within their voltage specifications.

2.3.2 Input Process:

- The primary input comes from the user's smartphone via Bluetooth communication.
- Commands sent from the smartphone are received by the HC-05 Bluetooth module connected to the STM32 microcontroller.
- The Bluetooth module converts the wireless signals into serial data to be processed by the microcontroller.

2.3.3 Microcontroller Processing:

- The STM32 microcontroller interprets the incoming data/commands.
- Based on the decoded commands, it sends appropriate control signals to the relay module.
- The microcontroller handles the timing, switching logic, and execution of commands to ensure accurate operation.

2.3.4 Output Process:

- The relay module receives data/control signals from the microcontroller.
- It switches the connected appliances (e.g., LED light, 5V fan) ON or OFF by connecting or disconnecting the power supply to these loads.
- The appliances respond immediately to the relay's switching status, completing the automation cycle.

2.3.5 Overall Operation:

- This system allows real-time control of home appliances wirelessly.
- Users can manage appliances remotely via Bluetooth without needing direct physical access or complex network setups.
- The integration provides a compact, efficient, and user-friendly smart home automation solution.

This structured operation ensures energy efficiency, improved convenience, and secure appliance management within the home environment.

2.4 Flow Chart (Algorithm)

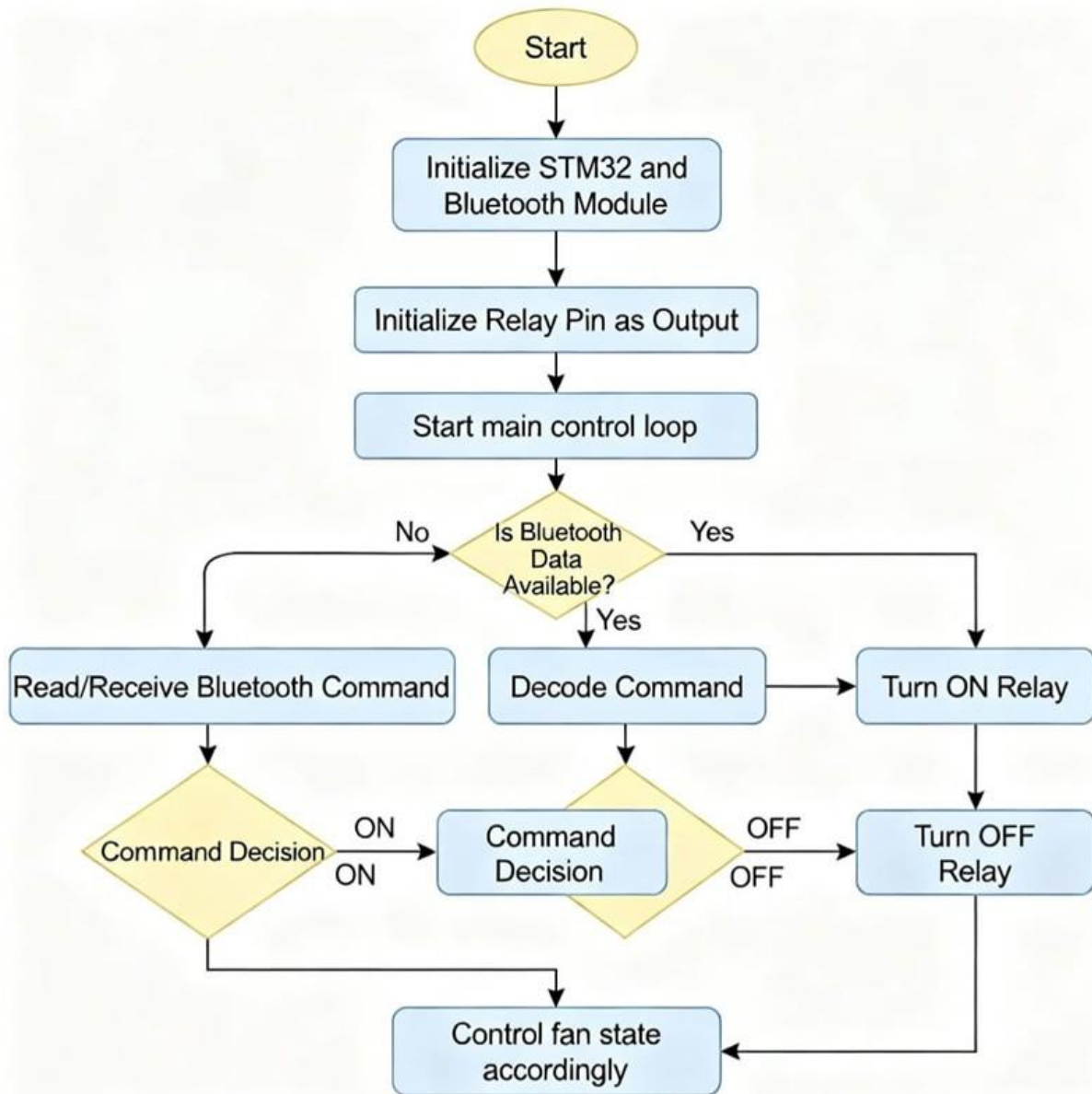


Figure 2.4[a]: Flow Chart

2.4.1 Smart Home Automation process flow - Algorithm

1. Start: The process begins.
2. Initialize STM32 and Bluetooth Module: The STM32 microcontroller and Bluetooth module (HC-05) are initialized and made ready for communication.

3. Initialize Relay Pin as Output: The GPIO pin connected to the relay is configured as an output pin (used to control the fan).

4. Start Main Control Loop: The program enters a continuous loop where it constantly checks for Bluetooth commands.

5. Check if Bluetooth Data is Available: The system checks whether any data (command) has been received via Bluetooth.

- If No: It waits to receive a Bluetooth command.
- If Yes: It reads and decodes the received command.

6. Decode Command: The received Bluetooth data is analyzed to identify whether it's an "ON" or "OFF" command.

7. Command Decision: Based on the decoded command:

- If the command is ON, the relay is turned ON (fan starts running).
- If the command is OFF, the relay is turned OFF (fan stops).

8. Control Fan State Accordingly: The fan's state (ON/OFF) is controlled according to the command received from the Bluetooth module.

CHAPTER 3

IMPLEMENTATION

3.1 Circuit Diagram

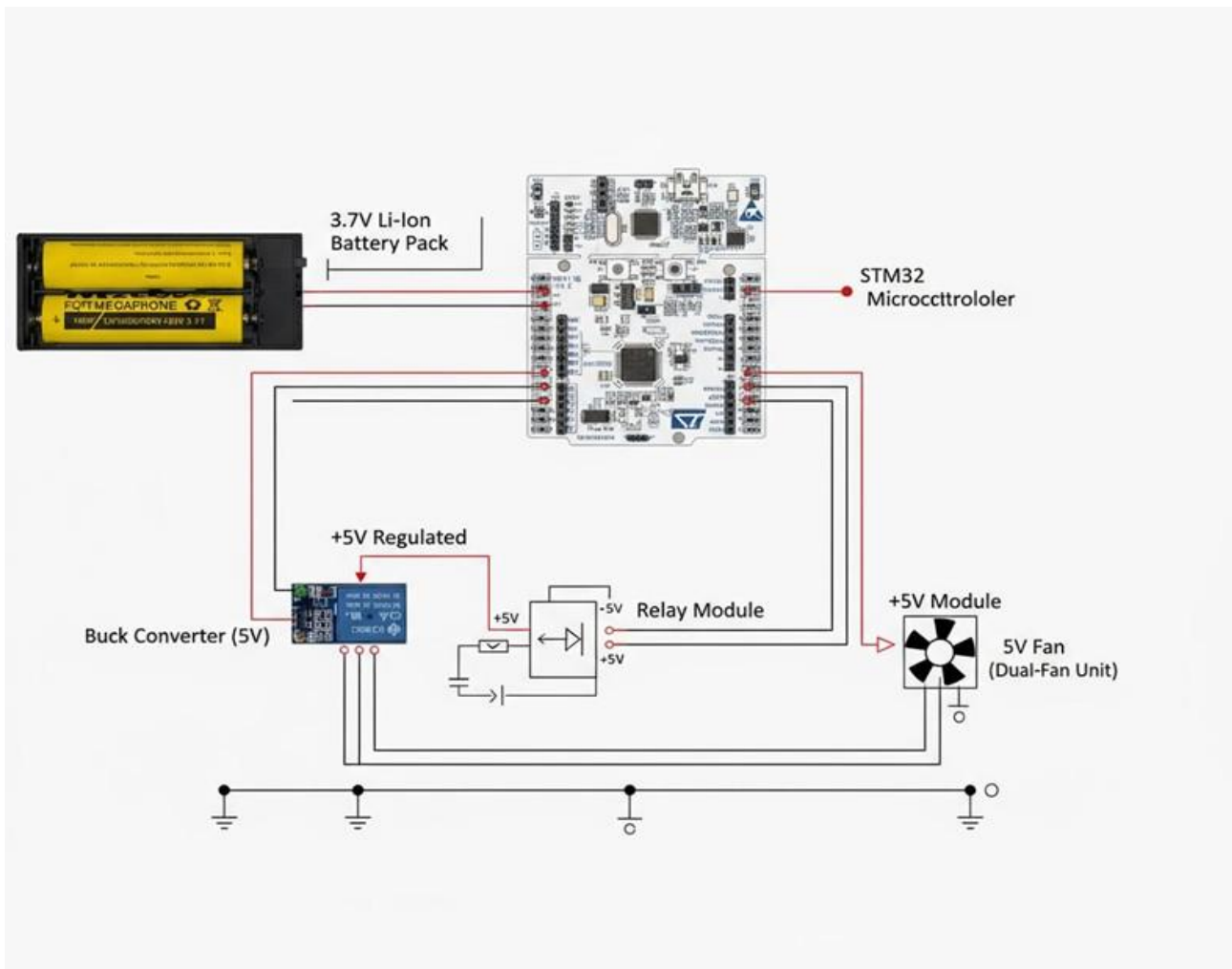


Figure 3.1[a]: Circuit Diagram

Circuit design methodology is a structured approach to designing electronic circuits that ensures that the circuit meets the required specifications and operates correctly. Here are the steps involved in circuit design methodology:

3.1.1 Requirement Analysis:

Before designing, the first step is to identify the goal and functional requirements of the circuit.

- Objective: To control a 5V fan using an STM32 microcontroller.
- Power Source: 3.7V Li-ion battery pack.
- Required Outputs: +5V regulated power for the relay and fan.
- Control: The STM32 should control the relay to switch the fan ON/OFF.

3.1.2 Power Supply Design:

The circuit needs different voltage levels:

- The 3.7V Li-ion battery provides the main input power.
- Since the STM32 and fan require 5V, a buck converter (step-up regulator) is used to convert 3.7V to +5V regulated output.
- Ensure proper grounding for all modules to maintain a common reference.

3.1.2 Microcontroller Interface Design:

- The STM32 microcontroller acts as the brain of the system.
- It takes input (control signals or program logic) and gives output to the relay module.
- The GPIO pin of STM32 is connected to the IN pin of the relay module to control its switching.
- The STM32 board itself is powered via the 3.7V battery connection.

3.1.3 Relay Module Connection:

- The relay module works as an electrically controlled switch.
- It receives a control signal (LOW/HIGH) from the STM32 and connects/disconnects the +5V supply to the fan
- The relay coil operates at +5V, powered from the buck converter output.

3.1.5 Load (Fan) Interface:

- The 5V DC fan (dual-fan unit) is connected as the load.

- One terminal of the fan connects to the relay's Normally Open (NO) contact, and the other to ground.
- When the relay is activated, +5V is applied to the fan, making it run.

3.1.6 Grounding and Common Reference:

All components (STM32, relay, buck converter, fan) share a common ground connection. This ensures proper signal referencing and stable operation.

3.1.7 Testing and Verification:

Once connections are made: Check voltage levels using a multimeter. Test relay switching through the STM32 output pin. Verify that the fan operates only when the STM32 sends a HIGH signal to the relay..

3.1.8 Summary of Flow:

Power Source (3.7V Battery) → Buck Converter (+5V Regulated) → Relay Module (Controlled by STM32) → Fan (+5V Output).

3.2 Components required

Both Hardware and Software components are required for the project.

3.2.1 Hardware requirements-

- STM32F446RETX Microcontroller
- HC-05 Bluetooth Module
- 3.7V Li-Ion Battery Pack
- Buck Converter Module for 5V Regulation
- Relay Module (5V)
- 5V LED Light and 5V Fan as Appliances
- Connecting Wires

- Breadboard
- Resistors and other supporting electronic components

3.2.2 Software requirements-

- STM32CubeIDE for firmware development
- ARM GCC or Keil Compiler for STM32
- Mobile App with Bluetooth control interface: Serial Bluetooth Terminal 1.49
- UART communication protocol implementation for Bluetooth
- Embedded C programming knowledge for control logic and peripheral initialization.

3.3 Code

```
#include "main.h"
```

```
UART_HandleTypeDef huart1;
```

```
UART_HandleTypeDef huart2;
```

```
uint8_t rx_data;    // to store received byte
```

```
uint8_t blink_flag = 0;
```

```
uint8_t fan_state = 0; // 0 = OFF, 1 = ON
```

```
void SystemClock_Config(void);
```

```
static void MX_GPIO_Init(void);
```

```
static void MX_USART2_UART_Init(void);
```

```
static void MX_USART1_UART_Init(void);
```

```
void do_action(uint8_t cmd) {
```

```
    switch (cmd) {
```

```
        case '1': // LED ON
```

```
            blink_flag = 0;
```

```
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
```

```
        HAL_UART_Transmit(&huart1, (uint8_t*) "LED ON\r\n", strlen("LED ON\r\n"),
HAL_MAX_DELAY);
        break;

    case '2': // LED BLINK until next command
        blink_flag = 1;
        HAL_UART_Transmit(&huart1, (uint8_t*) "LED BLINK\r\n", strlen("LED
BLINK\r\n"), HAL_MAX_DELAY);
        break;

    case '3': // LED OFF
        blink_flag = 0;
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
        HAL_UART_Transmit(&huart1, (uint8_t*) "LED OFF\r\n", strlen("LED
OFF\r\n"), HAL_MAX_DELAY);
        break;

    case '4': // FAN ON
        fan_state = 1;
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
        HAL_UART_Transmit(&huart1, (uint8_t*) "FAN ON\r\n", strlen("FAN ON\r\n"),
HAL_MAX_DELAY);
        break;

    case '5': // FAN OFF
        fan_state = 0;
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
        HAL_UART_Transmit(&huart1, (uint8_t*) "FAN OFF\r\n", strlen("FAN
OFF\r\n"), HAL_MAX_DELAY);
        break;
```

```
default:
    HAL_UART_Transmit(&huart1, (uint8_t*) "Invalid Option\r\n", strlen("Invalid
Option\r\n"), HAL_MAX_DELAY);
    break;
}
}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart->Instance == USART1) {

        char ch = (char) rx_data;

        /* Ignore CR/LF from terminal apps */
        if (ch == '\r' || ch == '\n') {
            HAL_UART_Receive_IT(&huart1, &rx_data, 1);
            return;
        }

        /* Valid commands: '1'...'5' */
        if (ch >= '1' && ch <= '5') {
            do_action(ch);
        } else {
            HAL_UART_Transmit(&huart1, (uint8_t*) "Invalid Option\r\n",
strlen("Invalid Option\r\n"), HAL_MAX_DELAY);
        }
        HAL_UART_Receive_IT(&huart1, &rx_data, 1);
    }
}
```

```
int main(void)
```

```
{

    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_USART2_UART_Init();
    MX_USART1_UART_Init();

    char menu[] =
        "Select Option:\r\n"
        "1. LED ON\r\n"
        "2. LED BLINK\r\n"
        "3. LED OFF\r\n"
        "4. FAN ON\r\n"
        "5. FAN OFF\r\n";
    HAL_UART_Transmit(&huart1, (uint8_t*) menu, strlen(menu), HAL_MAX_DELAY);

    HAL_UART_Receive_IT(&huart1, &rx_data, 1);

    if (blink_flag == 1) {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
        HAL_Delay(500);
    } else {
        HAL_Delay(10);
    }
}

void SystemClock_Config(void)
{
```



```
RCC_OscInitTypeDef RCC_OscInitStruct = {0};
RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

__HAL_RCC_PWR_CLK_ENABLE();

__HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE3);


RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
RCC_OscInitStruct.HSISState = RCC_HSI_ON;
RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
RCC_OscInitStruct.PLL.PLLM = 16;
RCC_OscInitStruct.PLL.PLLN = 336;
RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV4;
RCC_OscInitStruct.PLL.PLLQ = 2;
RCC_OscInitStruct.PLL.PLLR = 2;
if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
{
    Error_Handler();
}
RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                               |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;


if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
{
```

```
Error_Handler();
}
}
huart1.Instance = USART1;
huart1.Init.BaudRate = 9600;
huart1.Init.WordLength = UART_WORDLENGTH_8B;
huart1.Init.StopBits = UART_STOPBITS_1;
huart1.Init.Parity = UART_PARITY_NONE;
huart1.Init.Mode = UART_MODE_TX_RX;
huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
huart1.Init.OverSampling = UART_OVERSAMPLING_16;
if (HAL_UART_Init(&huart1) != HAL_OK)
{
    Error_Handler();
}

}

static void MX_USART2_UART_Init(void)
{

    huart2.Instance = USART2;
    huart2.Init.BaudRate = 115200;
    huart2.Init.WordLength = UART_WORDLENGTH_8B;
    huart2.Init.StopBits = UART_STOPBITS_1;
    huart2.Init.Parity = UART_PARITY_NONE;
    huart2.Init.Mode = UART_MODE_TX_RX;
    huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
    huart2.Init.OverSampling = UART_OVERSAMPLING_16;
    if (HAL_UART_Init(&huart2) != HAL_OK)
    {
```

```
Error_Handler();
}

}

static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    __HAL_RCC_GPIOC_CLK_ENABLE();
    __HAL_RCC_GPIOH_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    HAL_GPIO_WritePin(GPIOA, LD2_Pin|GPIO_PIN_6, GPIO_PIN_RESET);
    GPIO_InitStruct.Pin = B1_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(B1_GPIO_Port, &GPIO_InitStruct);

    GPIO_InitStruct.Pin = LD2_Pin|GPIO_PIN_6;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

}

void Error_Handler(void)
{
    __disable_irq();
    while (1)
```

```
{  
}
```

```
void assert_failed(uint8_t *file, uint32_t line)  
{
```

3.4 Explanation of the code

This program runs on an STM32F446RE Nucleo board.

It allows you to control an LED and a DC fan using UART communication (through Bluetooth or a serial terminal like Tera Term or the HC-05 module).

When you send numbers '1' to '5', it performs specific actions:

Command	Action
1	Turn LED ON
2	Make LED blink repeatedly
3	Turn LED OFF
4	Turn FAN ON
5	Turn FAN OFF

3.4.1 1. Includes and Declarations

```
#include "main.h"
```

```
UART_HandleTypeDef huart1;
```

```
UART_HandleTypeDef huart2;
```

```
uint8_t rx_data;
```

```
uint8_t blink_flag = 0;
```

```
uint8_t fan_state = 0;
```

- huart1, huart2: UART communication handlers.
- USART1 → connected to Bluetooth module (HC-05)
- USART2 → connected to the PC terminal (via USB)
- rx_data: stores one byte received from UART.
- blink_flag: used to decide if LED should blink
- fan_state: keeps track of fan ON/OFF..

3.4.2 do_action(uint8_t cmd) function

his function performs actions based on the received command (1–5).

Example:

```
case '1': // LED ON
```

```
    blink_flag = 0;
```

```
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
```

```
    HAL_UART_Transmit(&huart1, (uint8_t*) "LED ON\r\n", ...);
```

```
    break;
```

Explanation:

Turns LED ON (GPIO_PIN_5 = LED on Nucleo board)

Sends back message "LED ON" through UART

Similar for LED BLINK, LED OFF, FAN ON, FAN OFF.

GPIO_PIN_6 is used for the FAN output pin.

3.4.3 UART Receive Interrupt Callback

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
```

This function runs automatically whenever one byte is received through UART1.

It:

1. Reads the received character.
2. Ignores Enter or Newline (\r or \n).
3. Calls do_action() if it's a valid command ('1'–'5').
4. Re-enables UART receive interrupt so it keeps listening.

This ensures continuous communication without stopping the main loop.

3.4.4 Main Function

```
int main(void)
```

a. Initialization

```
HAL_Init();
```

```
SystemClock_Config();
```

```
MX_GPIO_Init();
```

```
MX_USART1_UART_Init();
```

```
MX_USART2_UART_Init();
```

Initializes hardware, system clock, GPIO pins, and UARTs.

b. Display Menu

```
char menu[] = "Select Option:\r\n1. LED ON\r\n2. LED BLINK\r\n...";
```

```
HAL_UART_Transmit(&huart1, (uint8_t*) menu, strlen(menu), HAL_MAX_DELAY);
```

Prints a command menu on Bluetooth terminal when program starts.

c. Start UART Reception

```
HAL_UART_Receive_IT(&huart1, &rx_data, 1);
```

Enables interrupt-based UART receiving (non-blocking).

3.4.5 Infinite Loop

```
while (1)
```

```
{
```

```
if (blink_flag == 1) {  
  
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);  
  
    HAL_Delay(500);  
  
} else {  
  
    HAL_Delay(10);  
  
}  
  
}
```

- If blink mode is ON → LED toggles every 500ms.
- If not → just waits shortly and continues checking.
- This allows blinking to happen in the background, while UART still works through interrupts.

3.4.6 System Clock & Peripheral Configuration

These functions (SystemClock_Config(), MX_USARTx_UART_Init(), MX_GPIO_Init()) are auto-generated by STM32CubeMX.

◆ SystemClock_Config()

Sets up internal oscillator (HSI) and PLL to get correct CPU frequency.

◆ MX_USART1_UART_Init()

Configures UART1:

Baud rate: 9600 (for Bluetooth HC-05)

8 data bits, 1 stop bit, no parity.

◆ MX_USART2_UART_Init()

Configures UART2:

Baud rate: 115200 (for PC serial monitor)

Often used for debugging messages.

- ◆ MX_GPIO_Init()

Initializes LED pin (PA5) and fan pin (PA6) as output pins.

3.4.7 Error Handler

```
void Error_Handler(void)
```

```
{
```

```
    __disable_irq();
```

```
    while (1) {}
```

```
}
```

If any HAL function fails, program stops here safely.

CHAPTER 4

HARDWARE AND SOFTWARE SPECIFICATIONS

4.1 Hardware Components Specifications

The required components are-

4.1.1 Microcontroller

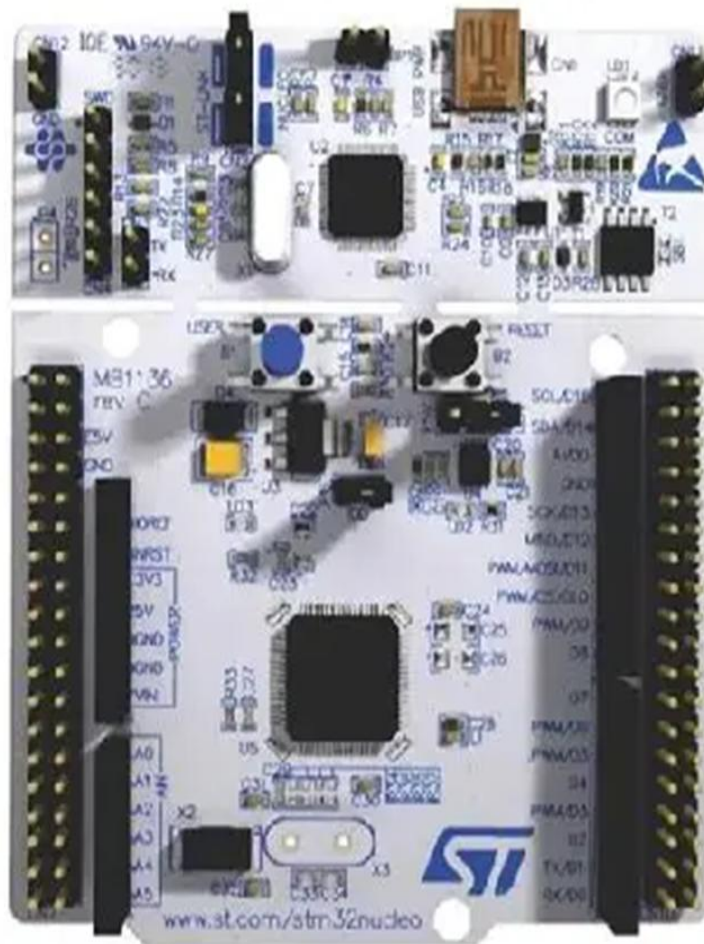


Figure 4.1.1[a]: STM32F446RETX board

The STM32F446RETx is a high-performance microcontroller from STMicroelectronics, based on the ARM Cortex-M4 core. The 'T' in the part number indicates a LQFP64 package.

The STM32F446RETx (specifically STM32F446RET6 in the LQFP64 package) boasts a powerful set of specifications:

- Core: ARM 32-bit Cortex-M4 CPU with Floating Point Unit (FPU) and Digital Signal Processing (DSP) instructions.
- Max Frequency: Up

Memory:

- Flash: 512 Kbytes.
- SRAM: 128 Kbytes + 4 Kbytes of backup SRAM.
- External Memory Interface: Flexible External Memory Controller (FMC) supporting SRAM, PSRAM, SDRAM/ LPDDR SDRAM, NOR/NAND Flash.
- Quad SPI Interface: Dual mode Quad SPI for external Flash memory.
- Operating Voltage: 1.7 V to 3.6 V.

Analog Peripherals:

- Analog-to-Digital Converters (ADC): Three 12-bit ADCs (up to 24 channels), 2.4 MSPS (up to 7.2 MSPS in triple interleaved mode).
- Digital-to-Analog Converters (DAC): Two 12-bit DAC channels.

Timers: Up to 17 timers, including:

- Two 32-bit general-purpose timers.
- Twelve 16-bit timers.
- Two advanced-control 16-bit timers (for motor control).

- Two watchdog timers and one SysTick timer.
- DMA: 16-stream DMA controller with FIFOs and burst support.
- I/O Ports: Up to 50 General-Purpose I/Os (GPIO) in the LQFP64 package, most of which are 5 V-tolerant.

4.1.1.1 Applications:

The extensive set of peripherals makes the STM32F446 suitable for high-performance and connected applications:

Connectivity:

- USB: USB 2.0 full-speed and high-speed device/host/OTG controller with an integrated full-speed PHY.
- CAN: Two CAN 2.0B Active interfaces.
- Serial Audio Interface (SAI): Two SAI interfaces.
- SDIO: Secure Digital Input/Output interface.
- SPDIF-Rx: Digital audio receiver interface.
- Camera Interface: 8- to 14-bit parallel camera interface.

Serial Communication:

- USART/UART: Four USARTs and two UARTs.
- SPI: Up to four SPI interfaces (up to 45\text{ Mbits/s}), three with multiplexed I2S.
- I²C: Up to four I²C interfaces (supporting SMBus/PMBus).
- Display: LCD parallel interface in 8080/6800 modes.

- Low Power Modes: Sleep, Stop, and Standby modes for power optimization.
- Clock Management: Internal (16 MHz HSI, 32 kHz LSI) and external (4–26 MHz HSE, 32.768 kHz LSE) clock sources with flexible PLL configuration.

4.1.1.2 Pin Description Summary (LQFP64 Package)

- The STM32F446RETx (LQFP64 package) has 64 pins, which are generally grouped by their primary function:

PIN TYPE	DESCRIPTION	KEY PINS/INTERFACES
Power Supply	Main power, analog power, and battery backup supply	VDD, VSS, VDDA, VSSA, VBAT
Clock/Reset	External oscillator connections and Reset input.	OSC_IN, OSC_OUT, NRST
GPIO Ports	General-purpose digital input/output, carrying multiple peripheral functions.	PA0–PA15, PB0–PB15, PC0–PC15, PD2, PH0, PH1
Analog	Pins primarily used for ADC/DAC functions.	ADC_INx, DAC_OUTx (These are multiplexed on GPIO pins like PA0, PA4, etc.)
Communication	Dedicated lines for major communication interfaces.	USARTx_Rx/Tx, SPIx_MISO/MOSI/SCK,

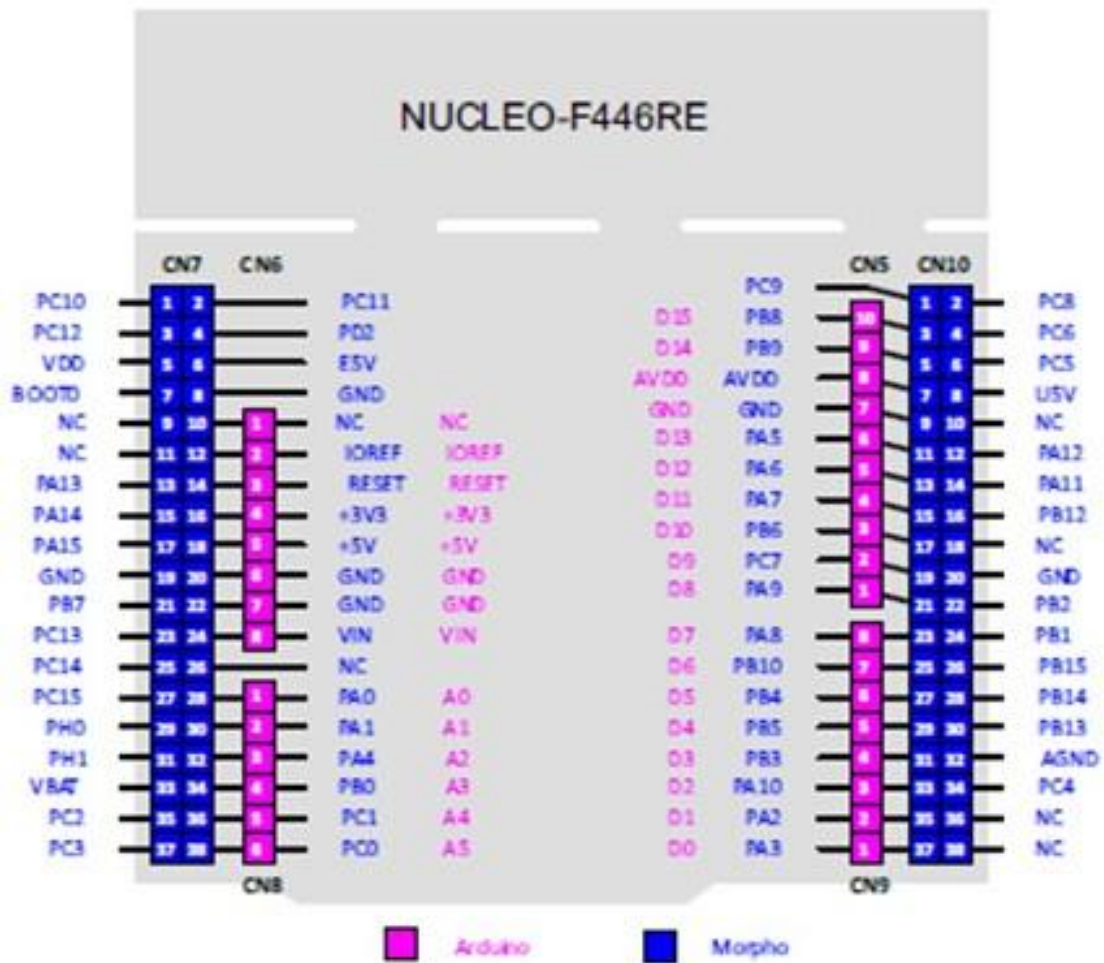


Figure 4.1.1.2[a]: Description of each pin of STM32F446RETX

4.1.2 . Bluetooth Module: HC-05

- Serial Bluetooth module for wireless communication.
- Bluetooth Version: 2.0+EDR (Enhanced Data Rate)
- Operating Voltage: 3.3V to 6V
- Communication Interface: UART (serial communication)
- Range: Up to 10 meters (approx. 33 feet) in open space (Class 2 Bluetooth)
- Baud Rates: Configurable from 9600 bps to 1382400 bps
- Operating Frequency: 2.4 GHz ISM band
- Modes: Master/Slave configurable
- Compatibility: Supports Android, Windows, and other Bluetooth-enabled devices

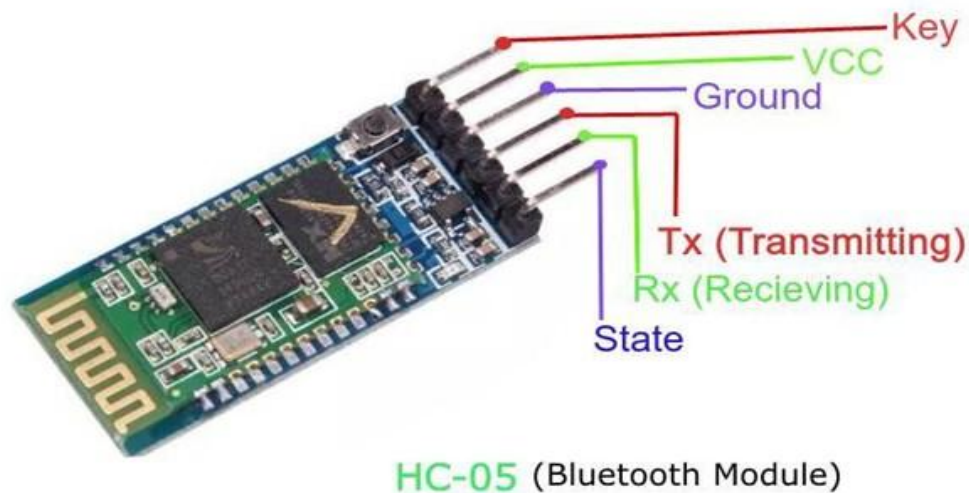


Figure 4.1.2[a]: HC-05 Module

4.1.3 Power Regulation: Buck Converter module

- Is used to step up and regulate 5V output from 3.7V battery.
- Converts higher voltage to a stable 5V output.
- Efficient DC-DC converter that maintains voltage regulation.
- Powers 5V components like relay module and fan.



Figure 4.1.3[a]: Buck Converter module

4.1.4 Relay Module 5v

- Electromechanical switch controlled by STM32 via 5V signal
- Isolates control signals from high-power appliances.
- Switches the 5V fan ON/OFF safely.
- Coil Voltage: 5V DC
- Switching Current: Usually up to 10A for appliances



Figure 4.1.4[a]:relay module

4.1.5 Load :Dual-Fan unit

- Typical low voltage fan used in cooling or ventilation.
- Powered directly by regulated 5V supply, switched via relay.



Figure 4.1.5[a]: Dual Fan-Unit

- **4.1.6 LED (Home Appliance Lighting Simulation):**

- Operating Voltage: 2V to 3.3V
- Forward Current: 10mA to 20mA
- Power Rating: Around 0.06W to 0.1W
- Purpose: Visual indicator and simulated home lighting control



Figure 4.1.6[a]:LED

4.1.7 7V Li-Ion Battery Pack

- Rechargeable lithium-ion battery providing a compact power source.
- Offers good energy density for portable applications.
- Supplies power to the entire circuit.



Figure 4.1.7[a]: 7V Li-Ion Battery Pack

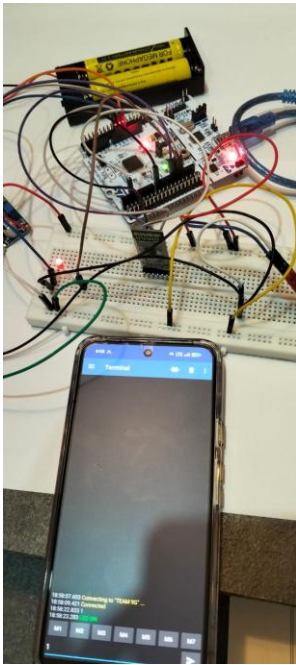
4.2 Software Specifications

- STM32CubeIDE for firmware development
- ARM GCC or Keil Compiler for STM32
- Mobile App with Bluetooth control interface: Serial Bluetooth Terminal 1.49
- UART communication protocol implementation for Bluetooth
- Embedded C programming knowledge for control logic and peripheral initialization.

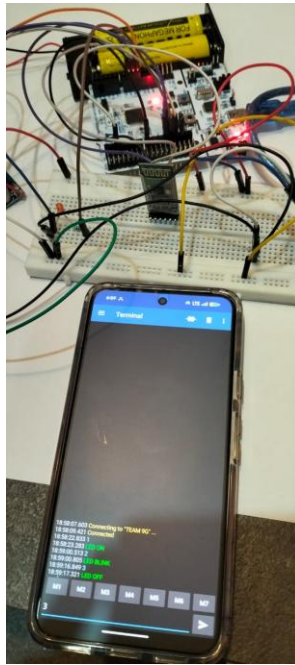
CHAPTER 5

RESULT AND DISCUSSION

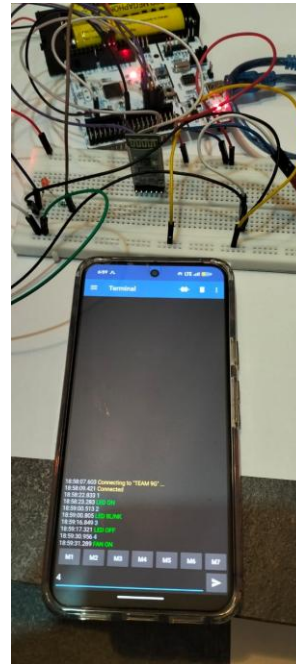
5.1 Result and Discussion



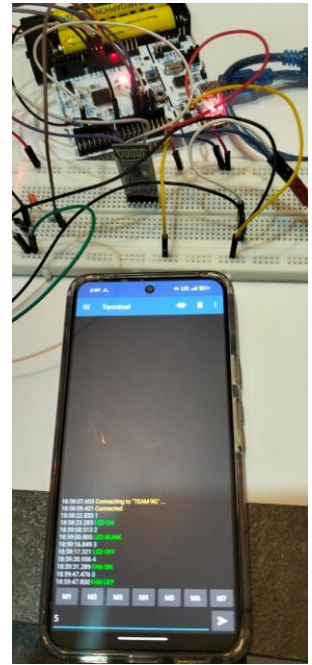
5.1[a]



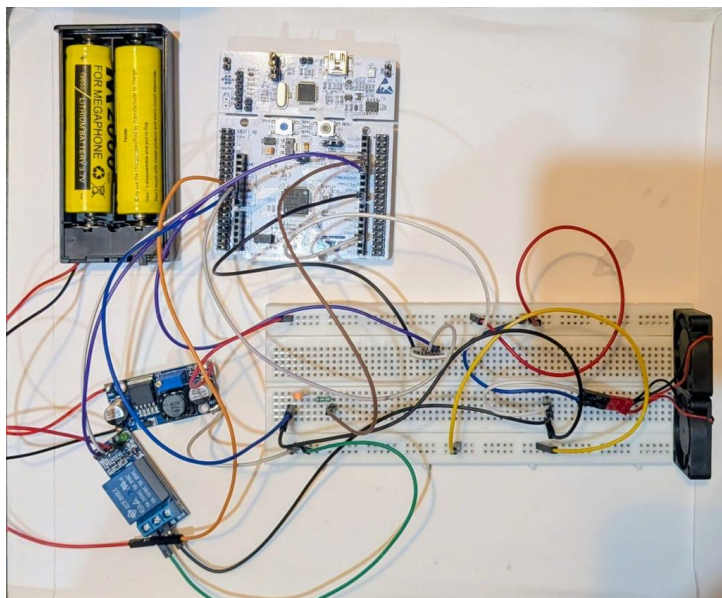
5.1[b]



5.1[c]



5.1[d]



5.1[e]

[a] =>command=1 =>LED ON

[b] => command=3 =>LED OFF

[c] =>command=4 =>FAN ON

[d] =>command =5 =>FAN OFF

[e] =>hardware interfacing

In this smart home automation project, the STM32 microcontroller with HC-05 Bluetooth module successfully enabled wireless control of home appliances, specifically the LED light and 5V fan.

Results:

- Reliable Bluetooth communication was established between the mobile device and the HC-05 module, allowing stable transmission of control commands.
- The STM32 decoded the incoming commands and accurately controlled the relay module to switch the LED light and fan ON/OFF.
- The buck converter provided a stable 5V regulated supply to the relay, fan, and other peripherals from the 3.7V Li-Ion battery, ensuring uninterrupted operation.
- The system demonstrated fast response times and consistent operation within the Bluetooth range (up to 10 meters).

Discussion:

The project effectively automates lighting and fan control, which are common home appliance functions, enhancing convenience and energy savings.

- By using Bluetooth-based control, the system avoids the need for complex wiring or internet connectivity, making it affordable and easy to implement.
- The relay module provides electrical isolation, improving safety when controlling high-power appliances.

- The rechargeable Li-Ion battery and efficient power management contribute to the system's portability and sustainability.

CHAPTER 6

CONCLUSION

This smart home automation project successfully demonstrates the use of an STM32 microcontroller and HC-05 Bluetooth module for wireless control of home appliances like LED lights and fans. The system provides a reliable and efficient solution for remote appliance control, enhancing convenience, safety, and energy savings. The use of a buck converter ensures stable power supply, and the relay module safely switches the appliances. This project is cost-effective, portable, user-friendly, and scalable, making it suitable for practical home automation applications as well as educational purposes in embedded systems and wireless technology.

CHAPTER 7

REFERENCES

- [1]STM32 Microcontroller Datasheet and Reference Manual - STMicroelectronics.
- [2]HC-05 Bluetooth Module User Manual and Specifications.
- [3]Buck Converter Module Datasheet – Manufacturer-specific details.
- [4]Relay Module Datasheets - Standard 5V relay module specifications.
- [5]Smart Home Automation using Bluetooth - Relevant academic papers and project guides.
- [6]STM32CubeIDE and Keil Development Tools documentation.
- [7]Various online tutorials and technical blogs on embedded home automation using STM32 and Bluetooth modules. These materials provide technical specifications,